to test CodeRunner programs, and installed a local sandbox server to test VPL so that the reader will be able to use sandbox servers online as well as locally. There is another reason why we do this. If you install more than one sandbox server in your local system, there will be a conflict because both the servers will use the same port address. Then you will have to manually change the port address for one of the servers. The advantage of using an online sandbox server is its ease of deployment whereas the disadvantage is that it is slow. If a large number of users take the test at the same time, the online sandbox server might deny service.

So, what we suggest is, experiment with both CodeRunner and VPL sandbox servers online, fix the plugin you like and set up the sandbox server of your choice locally so that faster execution is possible when a large number of students are taking the test. You can provide the address of the online Jobe server by taking the CodeRunner settings. The document *Configure.pdf* provides screenshots showing how to use the Jobe server installed in the local system as well as an online Jobe server.

Using CodeRunner

There are two different aspects to using plugins like CodeRunner. A programming language trainer will use CodeRunner to set an online programming test, whereas a student enrolled in that course will appear for this test to evaluate his or her level of knowledge. So, the two aspects of CodeRunner are setting up the test and taking the test.

We have already created a course titled *Programming* in Moodle. You should log in as an admin user, course designer or teacher, depending on the privileges provided by the admin user to create a CodeRunner test in Moodle. Here, in this example, we are logging in as the admin user to simplify things. The course called *Programming* will have a topic called Topic1, by default. Now, you need to create a quiz on Topic1 and add questions to it.

The questions can be first added to a question bank and then to the quiz. In our case, the name of the quiz is C to which we are adding a question type of CodeRunner. The *question type* option of CodeRunner allows a user to set the programming language in which the student is supposed to write the answer program. For this example, we have used *c_function* as the CodeRunner question type and thereby forced the student to write the answer as a C function. The other possible choices for CodeRunner question types include *c_program, cpp_ function, cpp_program, java_ class, java_method, java_program, php, python2, python3,* etc. Figure 2 shows the drop-down menu showing the question types of CodeRunner.

After setting this up, you need to provide details like the question name, question text, etc, so that the students get a clear idea regarding the answer they have to provide. Our question for this example is very simple — write a C programming language function to find the square of a number entered through the keyboard. The next task is to provide test cases for

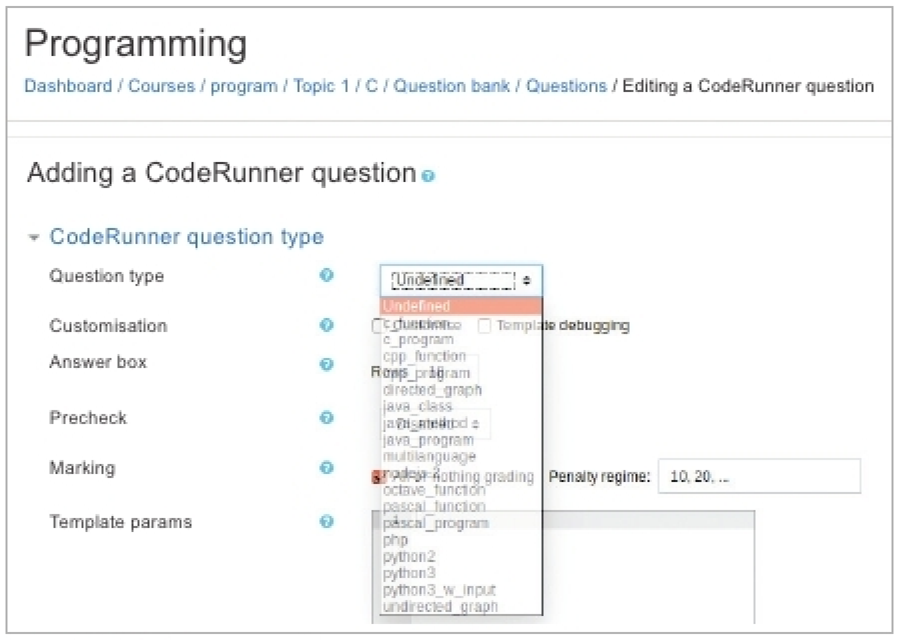

Figure 2: CodeRunner question types

automatic evaluation of the program provided by a student. For example, if the input is -11, the answer should be 121 and if the input is 5, the answer should be 25. It is better to give a large number of test cases to make sure that the program provided by the student is in fact correct. You should provide test cases involving corner cases to achieve this.

Now, you need to fix details regarding the grading and feedback for the test. Options like deferred feedback, adaptive mode, etc, can be used to modify the behaviour of the test. In our example, we have created only one question in the quiz called C to make things simpler, but remember there are no restrictions regarding the number of questions in a quiz. Now that we have created a test, student or guest users can attend this test depending on the privileges given to them. A user with proper authorisation can log in and attend this test. Figure 3 shows the environment provided by CodeRunner to write and execute answer programs. You will observe that some of the test cases are provided as examples to show the expected output.

Once the student users are fine with the program, they can test it with CodeRunner. There are options provided by CodeRunner to reduce the grade for a question with each

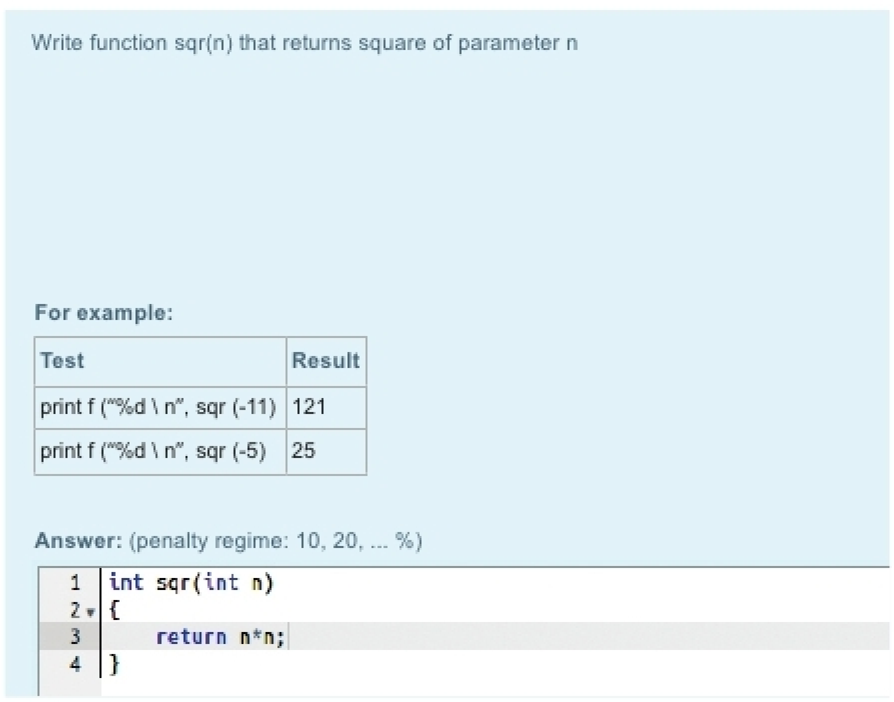

Figure 3: CodeRunner environment for program execution